Report on Assignment 2

MPI Programming

Students:

Sina Hatami
(5447389)
Behnoud Shafiezadeh Kenari
(5655740)
Shayan Alvansazyazdi
(5447411)

Feb 2024

1. Introduction

In this homework, we are going to implement the pi computation using MPI and will provide comparisons between different approaches to computations using processors and also in the sequential case.
The program has only one bottleneck which is represented below and the
loop here is dependent on the number of INTERVALS.
The aim here is to parallelize the program using MPI in different nodes of
the cluster to achieve a better performance in terms of runtime.

```
for (i = 1; i <= intervals; i++) {
x = dx * ((double) (i - 0.5));
f = 4.0 / (1.0 + x*x);
    sum = sum + f;
}
```

2. Sequential Case

To provide a better comparison we need to first run the program with the default number of INTERVALS which is 100000000000. Usually, it takes around some minutes for the program to finish the computation. To overcome this problem we will use the nodes available in the cluster and also the processors of each to parallelize in an efficient way to fasten the computation.

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
MPI_Comm_size(MPI_COMM_WORLD, &num_procs);

if (my_rank == 0) {
    printf("Number of intervals: %d\n", intervals);
}

// determine chunk size and start/end indices for each process
chunk_size = (intervals + num_procs - 1) / num_procs;
start = my_rank * chunk_size + 1;
end = (my_rank + 1) * chunk_size;
if (end > intervals) {
end = intervals;
}

For loop…

// combine partial results using MPI reduction operation
MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

Now, this part of the code calculates and shows the final calculation in the first rank:

```
// compute elapsed time and print results from rank 0
if (my_rank == 0) {
    time2 = MPI_Wtime();
    pi = dx*pi;
    printf("Computed PI %.24f\n", pi);
    printf("The true PI %.24f\n\n", PI25DT);
    printf("Elapsed time (s) = %.2lf\n", time2);
}

MPI_Finalize();
```

3. Result

This is the final results:

# Processors	1	2	4	8	16	32	64	128	256
Time (s)	4.7	4.4	4.2	4.3	2.1	2.2	2.1	2.2	2.2